

兆鸣嵌入式

C语言·一个LED讲透面向对象

EP09 你的全局变量，该死了

三家世界500强 · 11年一线嵌入式实战

公众号搜索「兆鸣嵌入式」回复「交流」进群

核心概念

每一份数据都要有主人。全局变量

=

没有主人的数据，谁都能碰、谁都能改，bug就藏在这种"公用"里。数据归位三步走：实例数据进struct，模块数据加static，常量用static const。

数据没有主人，bug就是主人

设计思路

问题：全局变量g_pin被第二次init覆盖，bug！

两个LED（红灯pin=5，绿灯pin=3）共享同一个g_pin全局变量。初始化绿灯时g_pin从5变成3——红灯的pin被覆盖！之后操作红灯，实际操作的是引脚3（绿灯的引脚）。这个bug非常隐蔽：代码不报错，逻辑全错。

解决：分三批归位

第一批：实例数据（pin, brightness）→ struct成员，跟着me指针走。红灯有红灯的pin，绿灯有绿灯的pin，互不干扰。

第二批：模块数据（init_count, debug_flag）→ static变量，文件私有，外面看不到。

第三批：只读常量（MAX_BRIGHTNESS）→ static const，有类型检查，不可修改。

视频讲义

你的 .c 文件开头

五个全局变量，从项目第一天就躺在这里

led.c 文件开头

```
/* led.c */
#include "led.h"
#include "platform.h"

int g_pin = 15;
int g_brightness = 100;
int init_count = 0;
int MAX_BRIGHTNESS = 255;
int g_debug_flag = 0;
```

△ 待处理

△ 待处理

△ 待处理

△ 待处理

△ 待处理

5 个全局变量 — 今天全部处理掉

兆鸣嵌入式

你的代码：5个全局变量散在文件开头

全局变量的陷阱

两个 LED 共享一个 g_pin — 第二次赋值覆盖第一次

Step 1

初始化红灯

led_init(red, pin=15);

// g_pin = 15 ✓

g_pin = 15 ✓

Step 2

初始化绿灯

led_init(green, pin=14);

// g_pin = 14 覆盖!

g_pin = 14 △ 覆盖了!

Step 3

点亮红灯

led_on(red);

// 操作 g_pin = 14
// 红灯不亮，绿灯亮了

红灯不亮，绿灯亮了!

g_pin 是全局的，只有一份 — 谁后赋值谁说了算

兆鸣嵌入式

Bug演示：第二次init覆盖g_pin

第一批

实例数据 → struct

红灯有红灯的，绿灯有绿灯的 — 属于实例的数据

Before 全局变量

两个 LED 共享，互相覆盖

```
int g_pin = 15;  
int g_brightness = 100;
```



搬进 struct

After struct 成员

每个 LED 实例自己带着

```
typedef struct {  
    int pin;  
    int brightness;  
} Led_t;
```

每个 LED 自己带着自己的数据 — 互不干扰

兆鸣嵌入式

第一批：实例数据搬进struct

第二批

模块私有 → static

不属于某个 LED，但也不该让外面看见

Before 裸全局变量

任何文件都能 extern 访问

```
int init_count = 0;  
int g_debug_flag = 0;
```



加 static

After 文件私有

关进本文件，外面碰不到

```
static int init_count = 0;  
static int debug_flag = 0;
```

 **static** = 部门内部的文件夹 — 部门外的人看不见

模块内部共享，但外面碰不到

兆鸣嵌入式

第二批：模块数据加static锁

第三批

常量 → static const

从来没人改过它 — 那就锁死

Before 可修改的变量

谁都能改，但从来没人改过

```
int MAX_BRIGHTNESS = 255;
```

➔
加 static const

After 编译器保护

出不了本文件，谁都改不了

```
static const int MAX_BRIGHTNESS = 255;
```

🔒 **static** = 出不了本文件 **const** = 谁都改不了 谁敢改，编译报错

#define 也行，但 static const 更好：类型明确，而且遵守作用域 — 出了本文件就看不见

只能看，不能改

兆鸣嵌入式

第三批：常量用static const上锁

改造前 vs 改造后

5 个变量，每个都找到了自己的位置

Before 原始代码

全局变量散落在文件开头

```
/* led.c */
```

```
#include "led.h"
```

```
int g_pin = 15;
```

→ struct

```
int g_brightness = 100;
```

→ struct

```
int init_count = 0;
```

→ static

```
int MAX_BRIGHTNESS = 255;
```

→ const

```
int g_debug_flag = 0;
```

→ static

➔

改造

After 改造后

每份数据都有了主人

```
/* led.c */
```

```
#include "led.h"
```

```
/* 常量 */
```

```
static const int MAX_BRIGHTNESS = 255;
```

```
/* 模块内部状态 */
```

```
static int init_count = 0;
```

```
static int debug_flag = 0;
```

```
/* 实例数据 */
```

```
typedef struct {
```

```
    int pin;
```

```
    int brightness;
```

```
} Led_t;
```

没有一个裸露的全局变量 — 清清爽爽

兆鸣嵌入式

改造后全貌：没有一个裸露的全局变量

数据归位表

以后遇到任何变量，看这张表

数据类型	去处	效果
实例数据	struct 成员	每个实例自己带着，跟着 me 走
模块私有	static	关进本文件，外面碰不到
只读常量	static const	谁都改不了，编译器保护
可写全局	尽量避免	99% 的时候，不需要

数据没有主人，bug 就是主人

兆鸣嵌入式

数据归位表：改造前 vs 改造后

static 的三种含义

同一个关键词，三种完全不同的意思

第一种

函数前加 static

```
static void do_something() {  
    // ...  
}
```

藏函数 — 这个函数只在本文件能用，别的文件找不到它

第二种★今天用的

变量前加 static

```
static int init_count = 0;
```

藏变量 — 这个变量只在本文件能用，和藏函数一个道理

第三种

局部变量加 static

```
void count() {  
    static int n = 0;  
    n++;  
}
```

延长寿命 — 函数结束它还活着，下次进来还是上次的值

活得比你想象的久，藏得比你想象的深

兆鸣嵌入式

static的三重含义

C 语言 vs C++：数据归属

C 语言你自己管，C++ 编译器帮你管

C 语言 你手动写

file-scope static + const

```
/* 模块私有共享数据 */
static int init_count = 0;

static = 本文件私有，别的文件看不见

/* 只读常量 */
static const int MAX = 255;
```

=

C++ 编译器帮你写

class 里的 static 成员

```
class Led {
    static int init_count;
};

static = 属于类，不属于实例，所有对象共享

class Led {
    static const int MAX = 255;
};
```

封装四期总结

✓ struct + me = class + this ✓ static + .h/.c = private + public ✓ 前缀 + init/deinit = 类名 + 构造/析构
✓ file-scope static = class static

封装，你学全了

兆鸣嵌入式

C vs C++ 对照

代码结构

文件	说明
led_bad.h	反面教材——全局变量满天飞的接口
led_bad.c	反面教材——5个全局变量，g_pin覆盖bug
led.h	正面教材——没有一个裸露的全局变量
led.c	正面教材——三类数据各归其位
main.c	演示反面 vs 正面对比 + 模块级数据
Makefile	编译脚本

关键代码解读

1. 反面教材——全局变量的g_pin覆盖bug

led_bad.c (反面教材)

```

/* led_bad.c -- 反面教材 */
int g_pin = 0;      /* 全局！任何人都能改！ */
int g_brightness = 0;
int init_count = 0; /* 没加static, 外部可见！ */
int MAX_BRIGHTNESS = 255; /* 用变量存常量？ */

int bad_led_init(uint8_t pin) {
    g_pin = pin; /* 每次init都覆盖！ */
    g_brightness = 0;
    init_count++;
    ...
}

```

两个LED共享g_pin，第二次init覆盖第一次。之后操作"红灯"实际操作的是绿灯的引脚。

2. 正面教材——三类数据各归其位

led.c (正面教材)

```

/* led.c -- 正面教材 */

/* 第三类：只读常量 */
static const uint8_t MAX_BRIGHTNESS = 100;
static const uint8_t MAX_PIN = 15;

/* 第二类：模块私有数据 */
static int s_init_count = 0;
static int s_debug_flag = 0;

/* 第一类：实例数据在struct里 (Led_t的成员) */
int led_init(Led_t *me, uint8_t pin) {
    me->pin = pin; /* 不是g_pin! */
    me->brightness = 0; /* 不是g_brightness! */
    s_init_count++; /* 模块级数据 */
    ...
}

```

改造后：.c文件开头没有一个裸露的全局变量。每一份数据都有主人。

数据归位表

改造前（全局变量）	改造后	归类
int g_pin	me->pin	实例数据 → struct成员
int g_brightness	me->brightness	实例数据 → struct成员
int init_count	static int s_init_count	模块数据 → static变量
int g_debug_flag	static int s_debug_flag	模块数据 → static变量
int MAX_BRIGHTNESS	static const uint8_t	常量 → static const

static的三重含义

用法	含义
static 修饰函数	文件私有函数（外部调不到）= C的private
static 修饰全局变量	文件私有变量（外部看不到）
static 修饰局部变量	值在函数调用间保持（不常用）

C vs C++ 对照

C 写法	C++ 写法
me->pin（struct成员）	this->pin（成员变量）
static int s_init_count	static int Led::init_count
static const uint8_t	static const成员 / constexpr
static void debug_print()	private方法
led_get_init_count()	Led::getInitCount()（静态方法）

编译运行

```
cd code/EP09_数据归位
make
./demo
```

获取更多

公众号搜索「兆鸣嵌入式」

回复「交流」加群，与一线工程师切磋

抖音 / B站 / 视频号

搜索「兆鸣嵌入式」